pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

Tuning the Pipeline: A Results Grid

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: a results grid

Twelve settings, six questions

Chunks	1,158 at 512/64, 520 at 1024/128
Retrieval	23 development questions, no request
Answers	6 development questions, saved runs
Requests	72 rows, 71 distinct
Held-out	not opened until day 11

Today

1. Names for settings
2. Expanding the context
3. What each setting sends
4. The grid, and reading it
5. Conservative sampling
6. Chunk size and depth together
7. Choosing on development questions only

Name every setting; write the rule before reading the table; six questions are few.

How a results grid works

The problem	A run that changes several settings at once cannot say which one moved the result
The method	A base record of every setting; each named setting changes the fewest fields; the same questions under every record; one table side by side
The hash	A record's hash names its row in the log, so one setting can never appear twice under two names
You control	Which fields to move, by how much, and which questions to run
What comes out	One row per setting, read for what moved, what changed nothing, and what each cost; no winner column

Twelve named settings

setting	what it changes in the base pipeline
k3	none: the base
k2	k=2
k5	k=5
chunk1024	size=1024, overlap=128
chunk1024_k2	size=1024, overlap=128, k=2
expand1	expand=1
hybrid	retrieve=hybrid
rerank	retrieve=hybrid, rerank=cross-encoder
temp03	temperature=0.3
temp07	temperature=0.7
top_p09	top_p=0.9
max_tokens150	max_tokens=150

temp0 is not a row: it is the base, one setting with two names and one hash.

Every field of RagSettings, and the settings that move it

field	default	base (k3)	part of the pipeline	moved by
`size`	`512`	`512`	chunker (day 3)	chunk1024, chunk1024_k2
`overlap`	`64`	`64`	chunker (day 3)	chunk1024, chunk1024_k2
`embedder`	`'minilm'`	`'minilm'`	embeddings (day 4)	no setting today
`index`	`'numpy'`	`'faiss'`	the index (day 4)	no setting today
`retrieve`	`'dense'`	`'dense'`	retrieval (day 6)	hybrid, rerank
`k`	`3`	`3`	retrieval (day 5)	k2, k5, chunk1024_k2
`candidates`	`20`	`20`	fusion and reranking (days 6, 7)	no setting today
`rerank`	`'none'`	`'none'`	reranking (day 7)	rerank
`expand`	`0`	`0`	context window (today)	expand1
`model`	`'gpt-4.1-mini'`	`'gpt-4.1-mini'`	the request (day 5)	no setting today
`temperature`	`0.0`	`0.0`	the request (day 5)	temp03, temp07
`top_p`	`1.0`	`1.0`	the request (day 5)	top_p09
`max_tokens`	`300`	`300`	the request (day 5)	max_tokens150

13 fields read from the dataclass; the grid moves 9 and leaves embedder, index, candidates, model at the base's value. A grid should say what it did not try.

How context-window expansion works

The problem	A retrieved chunk is often the right place cut off from the sentence before or after it that makes the figure readable
The method	Keep the retrieval; for each retrieved chunk join the n chunks before and after it in the same item, overlaps removed
What it keeps	The retrieved chunk's own citation; the same number of chunks, each longer
You control	n, the neighbours on each side (RagSettings.expand)
What comes out	More of the document's text in the message: the page rule cannot see it, the figure rule can

A context window around the match

483

characters, the retrieved chunk

1111

characters, with one neighbour each side

3

chunks joined, overlaps removed

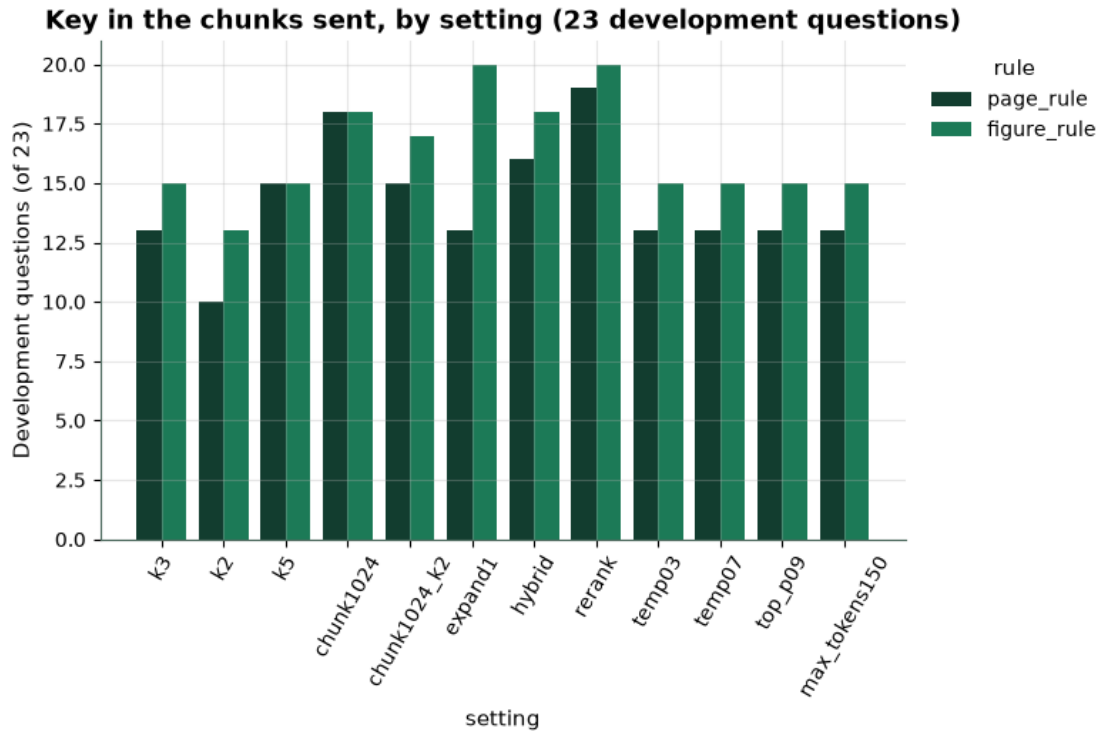
Chunks joined (dev13, top chunk)

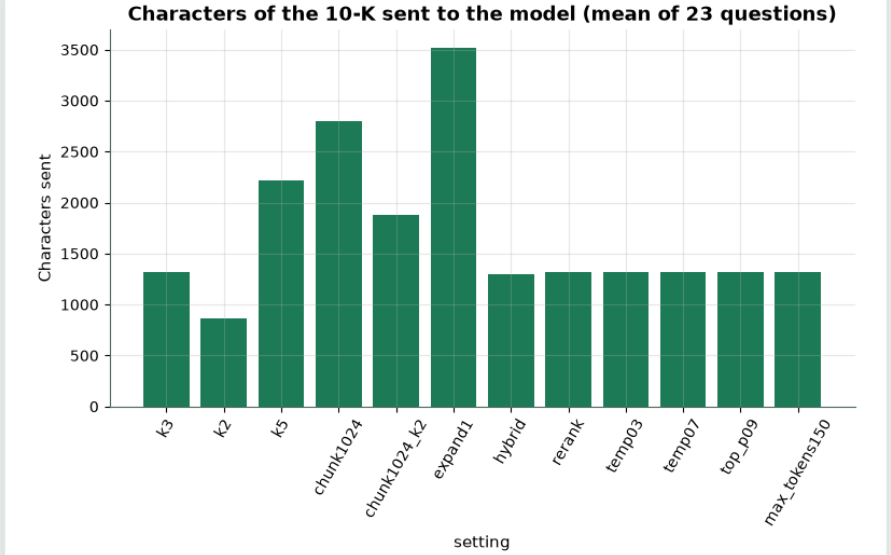
`boyd10k-8-059-0640, boyd10k-8-059-0641, boyd10k-8-059-0642`

The citation stays the retrieved chunk's; its neighbours are context.

Whether more text helps is a question for the grid, not for this cell.

What each setting sends



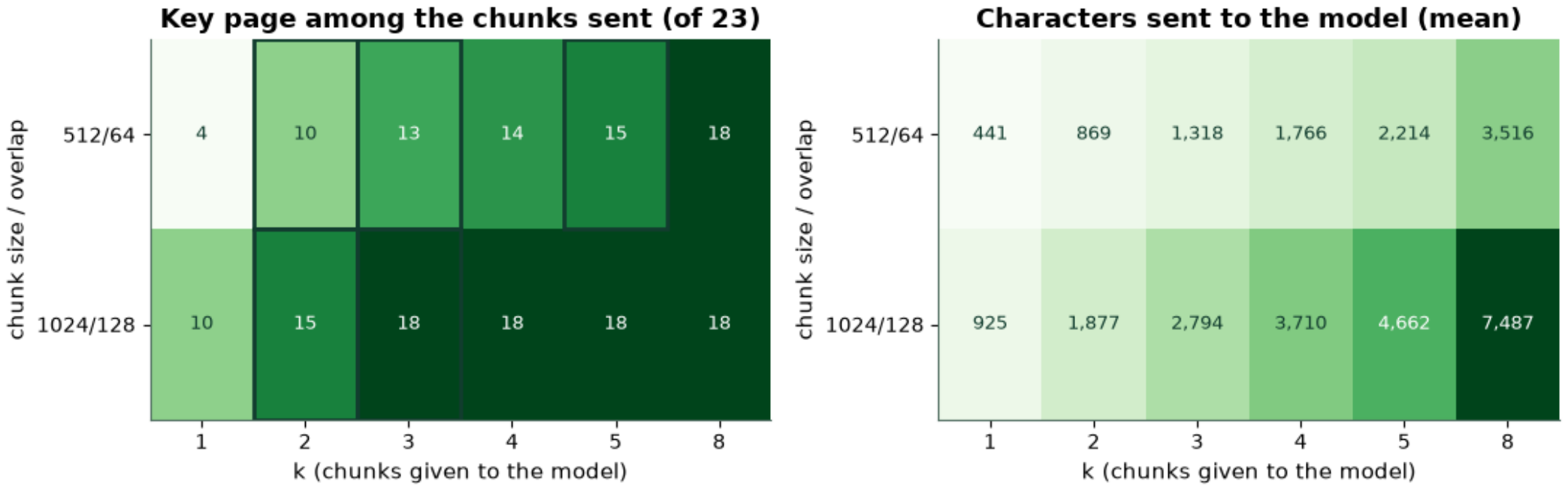


page rule: k3 13, k5 15, rerank 19

figure rule: k3 15, expand1 20

Of 23 development questions with a key page; no request is sent.

Two settings at once: chunk size against k




```
512/64 by k: 1 -> 4, 2 -> 10, 3 -> 13, 4 -> 14, 5 -> 15, 8 -> 18

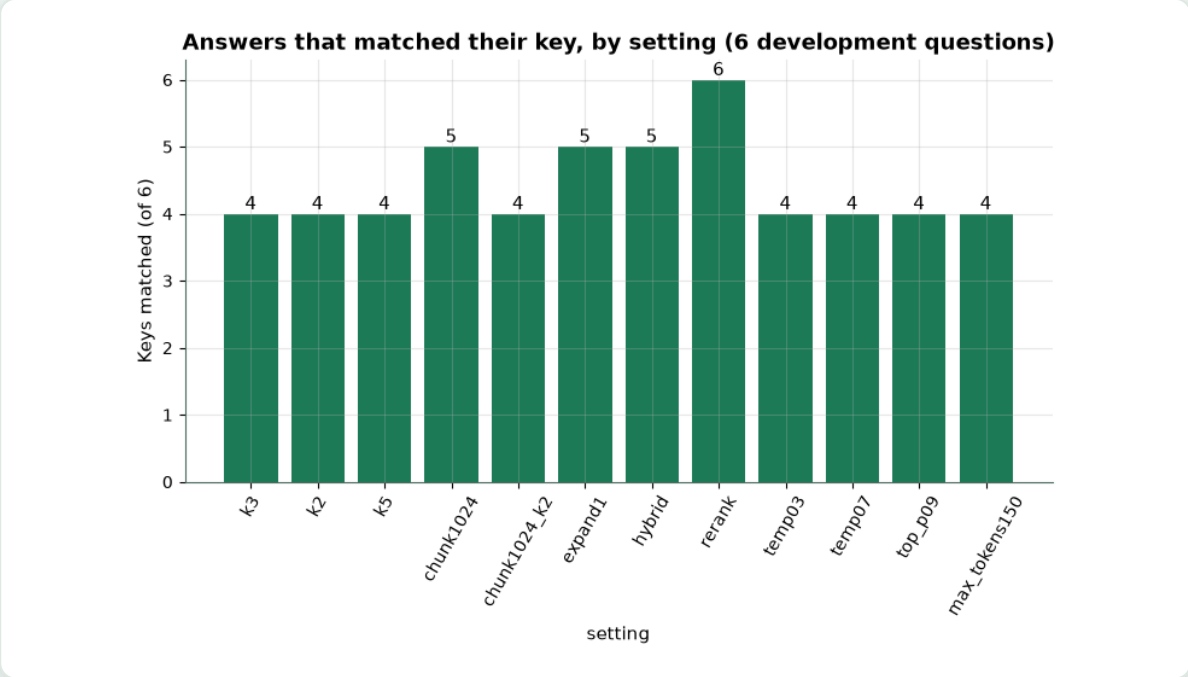
1024/128 by k: 1 -> 10, 2 -> 15, 3 -> 18, 4 -> 18, 5 -> 18, 8 -> 18
```

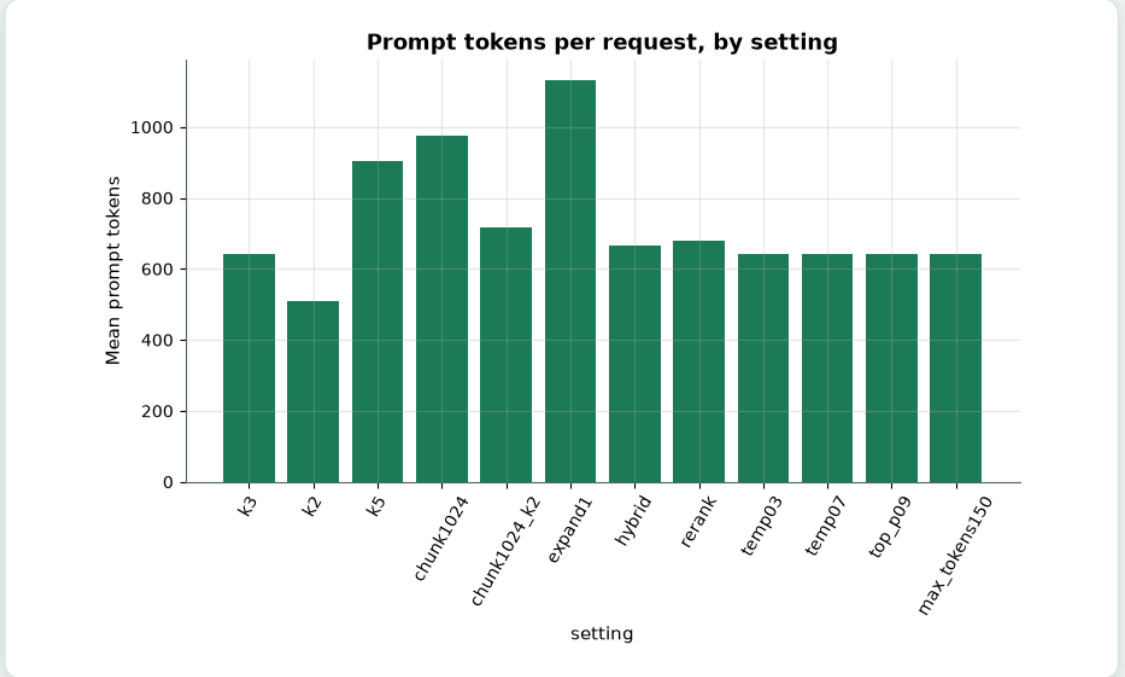
Twelve cells on 23 questions at no cost; the ringed five were answered. Read for a plateau, with the bill beside it.

How run_grid works

The problem	A grid run by hand is a loop that is easy to get wrong: a setting forgotten, a question answered twice, a held-out question slipped in
The method	For every setting build its retriever; for every question retrieve, build the prompt, send one request through the course's client, keep one row
The guards	A question outside the development split raises before any request; a request with no saved answer raises an error that names the setting
You control	The questions, the grid of records, and the mode (replay, or live with a key)
What comes out	Settings times questions rows; grid_summary folds them to one row per setting with no winner column

Twelve settings, six questions



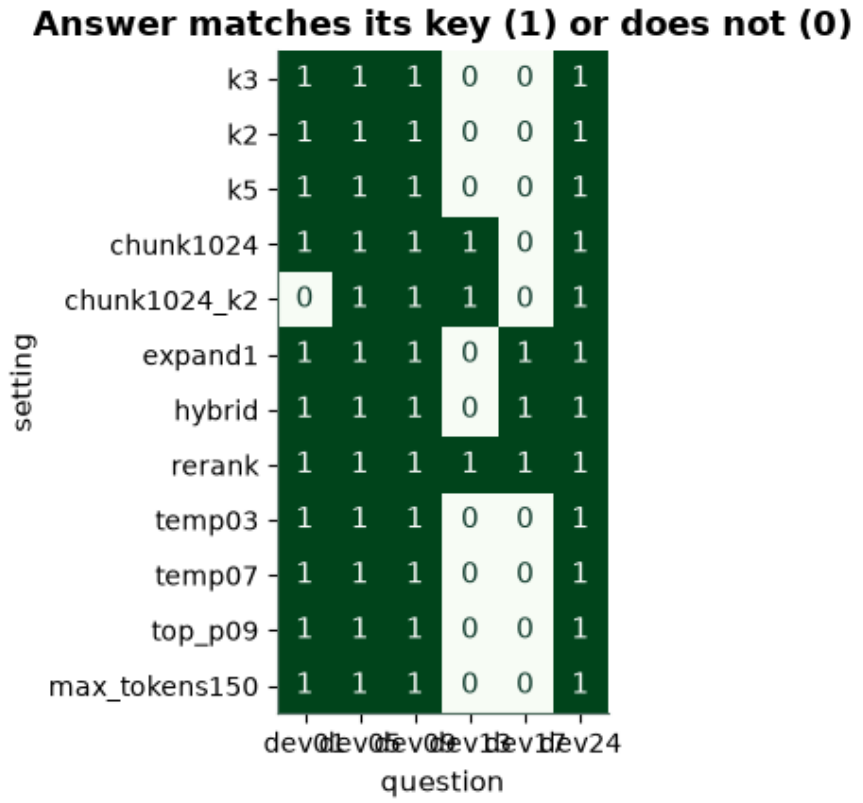


Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 72 requests, 52,746 prompt and 4,729 completion tokens; saved run recorded 2026-10-13; repla...

US$0.40 per million prompt tokens and US$1.60 per million completion tokens (developers.openai.com/api/docs/pricing, read 2026-10-13)

A count out of six moves one question at a time: one question is 17 points.

Which cells moved



Three readings

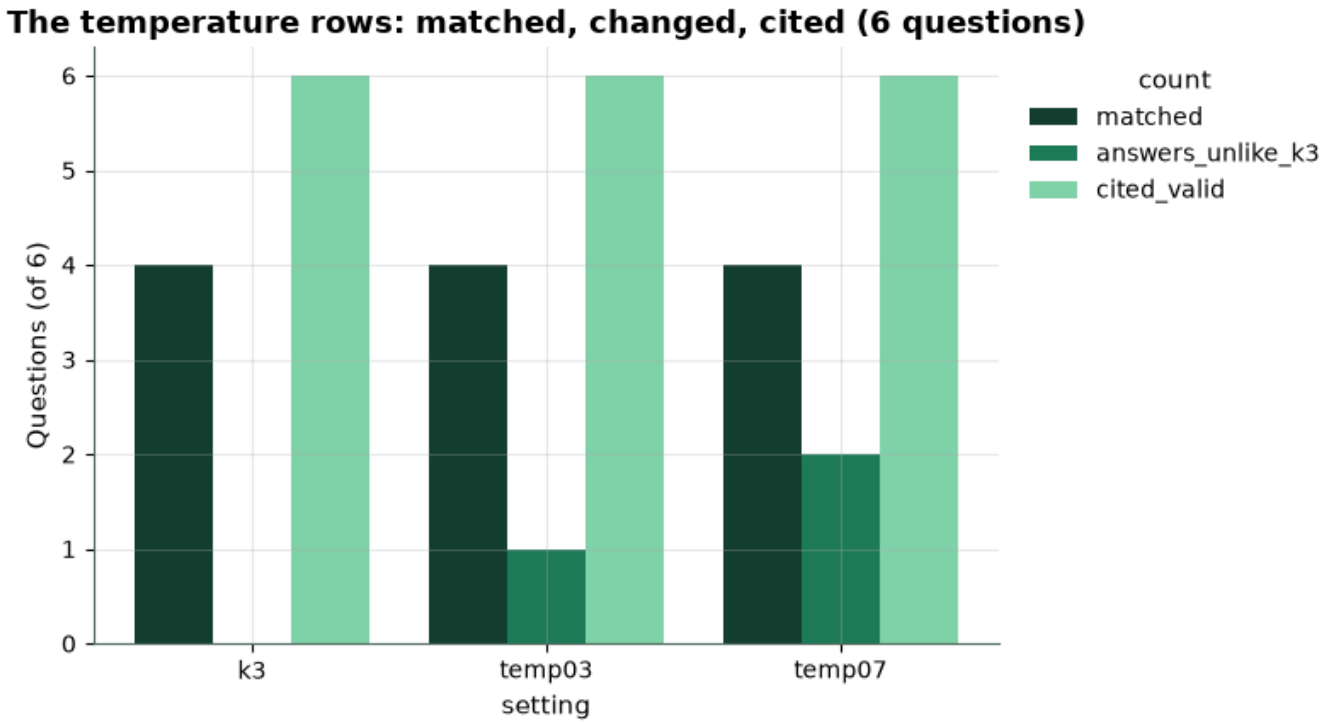
The settings differ on three of six questions: dev01, dev13, dev17. The other three do not depend on any setting here.

dev13: 8 settings answered 353,413 thousand, the right amount in the table's unit; the grader wants the key as written: no match.

Of 72 answers, 69 end every sentence with a citation; 23 hold the word will, quoted from the 10-K.

A changed text is not a changed grade; a grade measures a pipeline and a grader.

Conservative sampling, measured



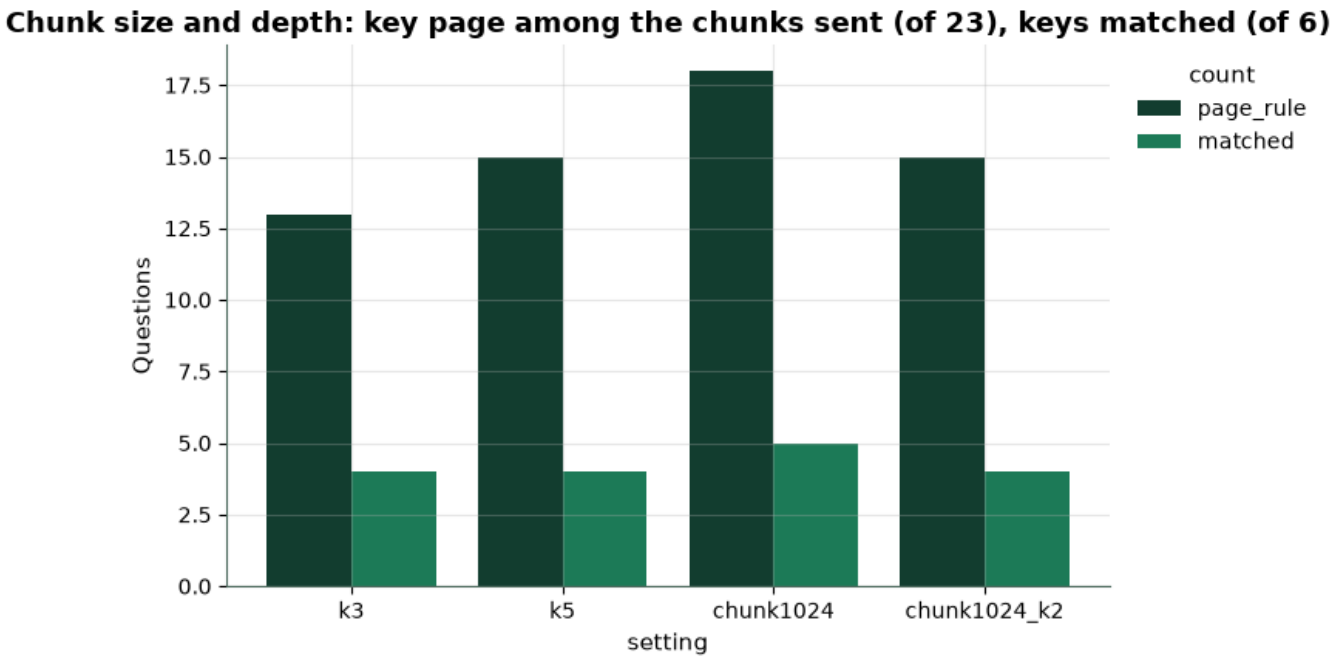
Six questions, one draw

temp	matched	unlike k3
0.0	4	0
0.3	4	1
0.7	4	2

No key was lost to the higher temperature. That is all six questions and one draw can say.

The SDK: alter temperature or top_p, but not both. The grid changes them in separate rows.

Chunk size and depth together



Characters sent, page rule, matched

k3 (512/64 at k = 3)

1,318 chars; page 13 of 23; matched 4 of 6

k5 (512/64 at k = 5)

2,214 chars; page 15 of 23; matched 4 of 6

chunk1024 (1024/128 at k = 3)

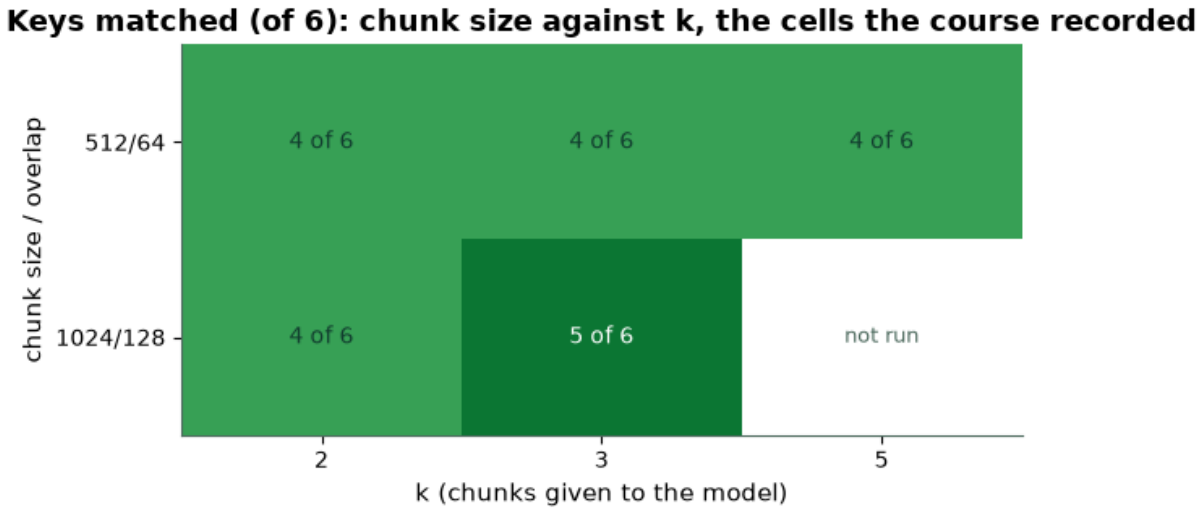
2,794 chars; page 18 of 23; matched 5 of 6

chunk1024_k2 (1024/128 at k = 2)

1,877 chars; page 15 of 23; matched 4 of 6

Liu and others, Lost in the Middle, arXiv 2307.03172 (2023-07-06; arXiv record read 13 October 2026). The paper itself was not read.

The answered grid over the same two settings



Five cells recorded, one not run

512/64 at k = 2 (k2) 4

512/64 at k = 3 (k3) 4

512/64 at k = 5 (k5) 4

1024/128 at k = 2 (chunk1024_k2) 4

1024/128 at k = 3 (chunk1024) 5

1024/128 at k = 5 was not run: the cell says so instead of showing a number.

The retrieval grid is twelve cells on 23 questions at no cost; the answered grid is five cells on six questions, paid for in requests. Sweep retrieval first, sample the answers from it.

How choosing on a grid stays honest

The problem	A rule invented after the table is read favours the row that happened to do well on these questions
The method	Write the rule first, in words that say exactly what it measures: the count, then the tie-breaks in order
The split	Development questions only, and the code refuses any other split; the held-out questions open once, later, for the one setting carried forward
You control	The rule, and nothing else once it is written
What comes out	One setting carried forward, described as what it is, never as the right pipeline

Choosing on development questions only

The rule, written first

1. Most keys matched, of 6

2. Tie: page rule on 23 questions

3. Tie: fewer prompt tokens

Development questions only

The twelve settings in the rule's order

setting	matched	page	tokens
rerank	6	19	678
chunk1024	5	18	975
hybrid	5	16	665
expand1	5	13	1133
chunk1024_k2	4	15	717
k5	4	15	905
k3	4	13	642
temp03	4	13	642
temp07	4	13	642
top_p09	4	13	642
max_tokens150	4	13	642
k2	4	10	509

Carried forward: rerank. The setting that matched the most keys on six development questions, not the optimal pipeline.

Excel side by side, and Git

Excel side by side (tested in Excel)

`=SUMIFS(runs[G],runs[A],A2)`	keys matched by rerank: 6, equal to Python
`=AVERAGEIFS(runs[G],runs[A],A2)`	share matched by k3: 0.6667
`=AVERAGEIFS(runs[G],runs[B],"512/64",runs[C],3)`	0.7500 over 48 rows
`=AVERAGEIFS(runs[H],runs[A],A2)`	mean prompt tokens, k3: 641.67
`=SORTBY(A2:A13,B2:B13,-1)`	the settings by keys matched: the same order

Git: rag_results.csv with 12 rows in two commits; 44 tests.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the filter keeps every question with a key page, so other splits get into the grid.

Code that runs is not the same as code that is right.

What you can do now

- Name every setting; differ from the base in one thing; list the fields the grid moves
- Count retrieval before paying for answers; sweep two settings as a heatmap
- run_grid refuses other splits and names the missing setting
- A changed text is not a changed grade; a cell not run stays blank
- Write the rule first; choose on development questions only

Week 2, Day 4 Metadata filters and many documents